```
/*
 * ============================================================
 *  Smart Security System
 * ============================================================
 *  Project     : Smart Security System
 *  Target HW   : Arduino Uno R3
 *  Libraries   : Adafruit_GFX, Adafruit_ST7735, SPI
 *
 * ------------------------------------------------------------
 *  Pin Assignment Table
 * ------------------------------------------------------------
 *  Signal         | Arduino Uno Pin  | Direction
 *  ----------------+------------------+------------------
 *  TFT CS         |      10        | OUTPUT
 *  TFT DC         |      9         | OUTPUT
 *  TFT RST        |      8         | OUTPUT
 *  TFT SDA (MOSI) |      11        | OUTPUT (HW SPI)
 *  TFT SCK (CLK)  |      13        | OUTPUT (HW SPI)
 *  HC-SR04 TRIG   |      5         | OUTPUT
 *  HC-SR04 ECHO   |      6         | INPUT
 *  Red LED        |      3         | OUTPUT
 *  Green LED      |      4         | OUTPUT
 *  Active Buzzer  |      7         | OUTPUT
 *  Push Button    |      2         | INPUT (INPUT_PULLUP)
 *
 * ------------------------------------------------------------
 *  Behavioural Summary
 * ------------------------------------------------------------
```

```
 *  The Smart Security System operates across three mutually
 *  exclusive states: DISARMED, ARMED, and ALARM. On power-up
 *  the system starts in DISARMED, indicated by a green LED and
 *  a "SYSTEM DISARMED" message on the 1.8-inch ST7735S TFT
 *  display. Pressing the push button arms the system (ARMED
 *  state), lighting the red LED and showing a live HC-SR04
 *  distance reading updated every 200 ms. If an object is
 *  detected within 50 cm continuously for 3 seconds, the system
 *  transitions to ALARM: the active buzzer produces a 300 ms ON /
 *  200 ms OFF siren, the red LED blinks at 250 ms intervals, and
 *  the display flashes an intruder alert between red and yellow
 *  at 500 ms. The alarm auto-silences after 30 seconds, returning
 *  to ARMED, or can be dismissed immediately by pressing the
 *  button, which returns to DISARMED. All timed operations use
 *  millis()-based non-blocking logic so the main loop never stalls.
 * ============================================================
 */

// — Library includes ————————————————————————

#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7735.h>

// — Pin assignments ————————————————————————

#define TFT_CS     10
#define TFT_DC      9
#define TFT_RST     8
#define PIN_TRIG    5
```

```
#define PIN_ECHO   6
#define PIN_LED_R  3
#define PIN_LED_G  4
#define PIN_BUZZER 7
#define PIN_BUTTON 2

// ── Timing constants (milliseconds) ──────────────────────
const unsigned long DEBOUNCE_INTERVAL_MS =  50UL;
const unsigned long SENSOR_POLL_MS       =  200UL;
const unsigned long GRACE_PERIOD_MS      = 3000UL;
const unsigned long ALARM_DURATION_MS    = 30000UL;
const unsigned long ALARM_FLASH_MS       =  500UL;
const unsigned long BUZZER_ON_MS         =  300UL;
const unsigned long BUZZER_OFF_MS        =  200UL;
const unsigned long LED_BLINK_MS         =  250UL;
const unsigned long SERIAL_DISTANCE_MS   =  500UL;

// ── Sensor thresholds ────────────────────────────────────
const float       DETECT_THRESHOLD_CM  =  50.0f;
const float       SENSOR_MAX_CM        = 400.0f;
const unsigned int  ECHO_TIMEOUT_US    = 30000U;  // 30 ms

// ── Display geometry ─────────────────────────────────────
const uint8_t TEXT_SIZE_STATUS  = 2;
const uint8_t TEXT_SIZE_DIST    = 2;

// ── State enumeration ────────────────────────────────────
enum SystemState : uint8_t {
```

```
  DISARMED = 0,
  ARMED    = 1,
  ALARM    = 2
};

// — TFT display object ————————————————————————
Adafruit_ST7735 tft(TFT_CS, TFT_DC, TFT_RST);

// — Global state variables ————————————————————
SystemState currentState;

// Button debounce
unsigned long lastButtonPressMs = 0;
bool        lastButtonReading = HIGH;

// Sensor polling
unsigned long lastSensorPollMs  = 0;
float       lastDistanceCm    = 0.0f;

// Grace period accumulation (start-time model)
unsigned long graceStartMs     = 0;
bool        graceActive      = false;

// Alarm pattern timers
unsigned long alarmEnteredMs    = 0;
unsigned long lastFlashMs       = 0;
unsigned long lastBuzzerMs      = 0;
unsigned long lastLedBlinkMs    = 0;
```

```
unsigned long lastSerialDistMs  = 0;

// Pattern state trackers
bool flashState    = false;
bool buzzerState   = false;
bool ledBlinkState = false;

// ── Function: renderDisplay ──────────────────────────────
// Clears the TFT and renders the full screen for the given state.
// Called by enterState() on every state transition.
void renderDisplay(SystemState state) {
  tft.fillScreen(ST7735_BLACK);        // clear artefacts from previous state
  tft.setTextWrap(false);

  switch (state) {

    case DISARMED:
      tft.setTextColor(ST7735_GREEN);
      tft.setTextSize(TEXT_SIZE_STATUS);
      tft.setCursor(5, 20);
      tft.print("SYSTEM");
      tft.setCursor(5, 50);
      tft.print("DISARMED");
      break;

    case ARMED:
      tft.setTextColor(ST7735_RED);
      tft.setTextSize(TEXT_SIZE_STATUS);
```

```
        tft.setCursor(5, 20);
        tft.print("SYSTEM");
        tft.setCursor(5, 50);
        tft.print("ARMED");
        // Initial distance placeholder — updated live by updateDistanceDisplay()
        tft.setTextColor(ST7735_WHITE);
        tft.setTextSize(TEXT_SIZE_DIST);
        tft.setCursor(5, 100);
        tft.print("Dist: -- cm");
        break;

      case ALARM:
        tft.setTextColor(ST7735_RED);
        tft.setTextSize(TEXT_SIZE_STATUS);
        tft.setCursor(5, 30);
        tft.print("! INTRUDER");
        tft.setCursor(5, 60);
        tft.print("ALERT!");
        break;
    }
}

// — measureDistance() ————————————————————————————
// Triggers the HC-SR04 ultrasonic sensor and converts the
// echo pulse duration to a distance in centimetres.
//
// Formula: distance (cm) = (duration_µs / 2) / 29.1
//   - Divide by 2 because the pulse travels to the target
```

```
//    and back (round trip).
//  - Divide by 29.1 because sound travels ~29.1 µs per cm
//    at room temperature (≈ 343 m/s).
//
// Returns 0.0f when pulseIn() times out (no echo received),
// which is used as an "invalid reading" sentinel by the caller.
float measureDistance() {
  // Send a clean 10 µs trigger pulse
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);

  // Measure the echo pulse width (µs); 0 = timeout
  unsigned long duration = pulseIn(PIN_ECHO, HIGH, ECHO_TIMEOUT_US);

  if (duration > 0) {
    return (duration / 2.0f) / 29.1f;   // µs → cm
  }
  return 0.0f;   // timeout / invalid sentinel
}

// — updateDistanceDisplay() ————————————————————
// Erases only the distance line on the TFT (y = 95–120) and
// redraws the current reading, avoiding a full fillScreen()
// which would flicker the status text above it.
//
```

```
// Requirements: 4.5
void updateDistanceDisplay(float cm) {
  // Erase previous distance value — filled black rectangle
  // covers y=95 to y=120 (25 px tall), full display width
  tft.fillRect(0, 95, 128, 25, ST7735_BLACK);

  // Redraw distance value at same position
  tft.setCursor(5, 100);
  tft.setTextColor(ST7735_WHITE);
  tft.setTextSize(TEXT_SIZE_DIST);
  tft.print("Dist: ");
  tft.print(cm, 1);  // 1 decimal place
  tft.print(" cm");
}

// — Function: checkButton ————————————————————————
// Reads the push button (INPUT_PULLUP; LOW = pressed) and
// fires handleButtonPress() on a debounced falling edge.
//
// Debounce logic:
//  A falling edge is confirmed only when the gap since the
//  previous confirmed press is ≥ DEBOUNCE_INTERVAL_MS (50 ms).
//  This suppresses contact bounce without blocking the loop.
void checkButton() {
  bool currentReading = digitalRead(PIN_BUTTON);

  // Falling edge: button just went from HIGH (released) to LOW (pressed)
  if (currentReading == LOW && lastButtonReading == HIGH) {
```

```
      unsigned long now = millis();
      if ((now - lastButtonPressMs) >= DEBOUNCE_INTERVAL_MS) {
        lastButtonPressMs = now;
        handleButtonPress();
      }
      // else: within debounce window — discard the transition
    }

    lastButtonReading = currentReading;
}

// — handleButtonPress() ————————————————————————
// Called by checkButton() on every confirmed (debounced) press.
// Logs the event to Serial, then drives the state machine to
// the correct next state based on the current state:
//   DISARMED → ARMED
//   ARMED    → DISARMED
//   ALARM    → DISARMED  (also clears grace timer via enterState)
void handleButtonPress() {
    Serial.println("[BTN] Button pressed");

    switch (currentState) {
      case DISARMED:
        enterState(ARMED);
        break;

      case ARMED:
        enterState(DISARMED);
```

```
        break;

      case ALARM:
        enterState(DISARMED);
        break;
  }
}

// — enterState() ——————————————————————————————
// Central state-transition function. Every state change in the
// system must go through this function — never assign currentState
// directly elsewhere.
//
// Sequence:
//  1. Log the transition to Serial.
//  2. Reset all shared outputs (buzzer, LEDs, grace timer,
//     pattern booleans) so no artefacts carry into the new state.
//  3. Render the TFT screen for the new state.
//  4. Apply state-entry actions (record timers, set LEDs).
//  5. Commit the new state.
//
// Requirements: 1.1, 1.2, 1.3, 1.4, 3.1–3.4, 4.1–4.4,
//           6.4, 7.1–7.5, 8.3, 10.2
void enterState(SystemState next) {

  // — 1. Serial transition log ——————————————————
  auto stateName = [](SystemState s) -> const char* {
    switch (s) {
```

```
      case DISARMED: return "DISARMED";
      case ARMED:    return "ARMED";
      case ALARM:    return "ALARM";
      default:       return "UNKNOWN";
    }
};

Serial.print("[STATE] ");
Serial.print(stateName(currentState));
Serial.print(" -> ");
Serial.println(stateName(next));

// — 2. Reset shared outputs ——————————————————
// Ensure buzzer and both LEDs are off before the new
// state's entry actions set them to the correct level.
digitalWrite(PIN_BUZZER, LOW);
digitalWrite(PIN_LED_R,  LOW);
digitalWrite(PIN_LED_G,  LOW);

// Clear grace period so a stale window from ARMED does
// not persist if we return to ARMED via auto-silence.
graceActive  = false;
graceStartMs = 0;

// Reset alarm-pattern booleans so patterns always start
// from a known state when (re-)entering ALARM.
flashState   = false;
buzzerState  = false;
```

```
ledBlinkState = false;

// ── 3. Render TFT screen for the new state ──────────────
renderDisplay(next);

// ── 4. State-entry actions ──────────────────────────────
unsigned long now = millis();

switch (next) {

  case ALARM:
    // Record when the alarm started so auto-silence can
    // measure elapsed time, and seed pattern timers so
    // the first toggle fires on schedule.
    alarmEnteredMs = now;
    lastFlashMs    = now;
    lastBuzzerMs   = now;
    lastLedBlinkMs = now;
    // Buzzer starts in the ON phase immediately.
    digitalWrite(PIN_BUZZER, HIGH);
    buzzerState = true;
    break;

  case ARMED:
    // Red LED on continuously while monitoring.
    digitalWrite(PIN_LED_R, HIGH);
    break;
```

```
      case DISARMED:
        // Green LED on continuously while disarmed.
        digitalWrite(PIN_LED_G, HIGH);
        break;
    }

    // ── 5. Commit new state ──────────────────────────────
    currentState = next;
}

// ── updateAlarmPatterns() ──────────────────────────────────
// Drives three independent non-blocking millis() timers while
// the system is in ALARM state.  Called every loop() iteration
// by updateStateMachine() when currentState == ALARM.
//
//  1. TFT flash      – toggles text colour between ST7735_RED
//                      and ST7735_YELLOW every 500 ms (ALARM_FLASH_MS).
//  2. Buzzer siren   – 300 ms ON (BUZZER_ON_MS) / 200 ms OFF
//                      (BUZZER_OFF_MS) duty cycle.
//  3. Red LED blink  – toggles the red LED every 250 ms (LED_BLINK_MS).
//
// Requirements: 6.1, 6.2, 6.3, 12.5
void updateAlarmPatterns() {
    unsigned long now = millis();

    // ── 1. TFT flash — 500 ms colour toggle ──────────────
    if (now - lastFlashMs >= ALARM_FLASH_MS) {
        flashState = !flashState;
```

```
    uint16_t color = flashState ? ST7735_YELLOW : ST7735_RED;
    // Erase text cells with a black rect then redraw with the new colour.
    // fillRect() covers y = 25 to y = 75 (both text lines), avoiding
    // a full fillScreen() which would cause visible flicker.
    tft.fillRect(0, 25, 128, 50, ST7735_BLACK);
    tft.setTextColor(color);
    tft.setTextSize(TEXT_SIZE_STATUS);
    tft.setCursor(5, 30);
    tft.print("! INTRUDER");
    tft.setCursor(5, 60);
    tft.print("ALERT!");
    lastFlashMs = now;
  }

  // — 2. Buzzer siren — 300 ms ON / 200 ms OFF ——————
  // The period varies depending on the current phase so the
  // ON and OFF durations are asymmetric (siren cadence).
  unsigned long buzzerPeriod = buzzerState ? BUZZER_ON_MS : BUZZER_OFF_MS;
  if (now - lastBuzzerMs >= buzzerPeriod) {
    buzzerState = !buzzerState;
    digitalWrite(PIN_BUZZER, buzzerState ? HIGH : LOW);
    lastBuzzerMs = now;
  }

  // — 3. Red LED blink — 250 ms toggle ————————————
  if (now - lastLedBlinkMs >= LED_BLINK_MS) {
    ledBlinkState = !ledBlinkState;
    digitalWrite(PIN_LED_R, ledBlinkState ? HIGH : LOW);
```

```
      lastLedBlinkMs = now;
    }
}

// — updateStateMachine() ————————————————————————
// Called every loop() iteration. Evaluates state-specific
// timed logic without blocking:
//
//   DISARMED: nothing — waits passively for a button event.
//
//   ARMED: if the grace period is active, checks whether
//     GRACE_PERIOD_MS (3 s) of uninterrupted detection has
//     elapsed since graceStartMs and transitions to ALARM.
//     Uses the start-time model (see design.md §Data Models).
//
//   ALARM: delegates to updateAlarmPatterns() for buzzer /
//     LED / TFT toggle timers, then checks the 30-second
//     auto-silence timer and transitions back to ARMED when
//     ALARM_DURATION_MS has elapsed.
//
// Requirements: 5.1, 5.2, 5.3, 6.5, 7.1, 7.2, 7.3, 10.4, 12.5
void updateStateMachine() {
  switch (currentState) {

    case DISARMED:
      // No periodic logic required — state changes only via button press.
      break;
```

```
      case ARMED:
        if (graceActive) {
          unsigned long elapsed = millis() - graceStartMs;
          Serial.print("[GRACE] Elapsed: ");
          Serial.print(elapsed);
          Serial.println(" ms");
          if (elapsed >= GRACE_PERIOD_MS) {
            enterState(ALARM);
          }
        }
        break;

      case ALARM:
        updateAlarmPatterns();
        if (millis() - alarmEnteredMs >= ALARM_DURATION_MS) {
          enterState(ARMED);
        }
        break;
    }
}

// — updateSensor() ————————————————————————————
// Called every loop() iteration. Fires the HC-SR04 sensor
// every SENSOR_POLL_MS (200 ms) while in ARMED state and
// maintains the grace-period accumulation logic.
//
// Distance classification:
//   0.0f                → INVALID  (pulseIn timeout)
```

```
//   0.0f < d ≤ DETECT_THRESHOLD_CM (50 cm) → DETECTION
//   DETECT_THRESHOLD_CM < d ≤ SENSOR_MAX_CM (400 cm) → NO DETECTION
//   d > SENSOR_MAX_CM        → INVALID  (out of range)
//
// Grace period uses a "start-time" model: a fresh window opens
// on the first detection tick; any non-detection or invalid
// reading resets the window immediately.
//
// Requirements: 3.5, 4.6, 5.1, 5.2, 5.4, 10.4
void updateSensor() {
  // ── Guard 1: only poll in ARMED state ──────────────────
  if (currentState != ARMED) {
    return;
  }

  // ── Guard 2: throttle to one measurement per 200 ms ────
  unsigned long now = millis();
  if ((now - lastSensorPollMs) < SENSOR_POLL_MS) {
    return;
  }

  // ── 3. Record the timestamp of this poll ───────────────
  lastSensorPollMs = now;

  // ── 4. Acquire distance reading ────────────────────────
  lastDistanceCm = measureDistance();

  // ── 5. Refresh live distance on TFT ────────────────────
```

```
  updateDistanceDisplay(lastDistanceCm);

  // — 7. Periodic Serial distance log (every 500 ms) ————
  if ((now - lastSerialDistMs) >= SERIAL_DISTANCE_MS) {
    Serial.print("[DIST] ");
    Serial.print(lastDistanceCm, 1);
    Serial.println(" cm");
    lastSerialDistMs = now;
  }

  // — 6. Grace-period accumulation (start-time model) ——
  bool isDetection  = (lastDistanceCm > 0.0f && lastDistanceCm <=
DETECT_THRESHOLD_CM);
  bool isNoDetect   = (lastDistanceCm > DETECT_THRESHOLD_CM && lastDistanceCm
<= SENSOR_MAX_CM);
  // Anything else (== 0.0f or > SENSOR_MAX_CM) is INVALID

  if (isDetection) {
    // Object in range — open a new window or keep the existing one
    if (!graceActive) {
      graceActive  = true;
      graceStartMs = millis();
    }
    // Note: transition to ALARM on elapsed >= GRACE_PERIOD_MS is
    // evaluated by updateStateMachine(), not here, to keep
    // sensor polling and state transitions cleanly separated.
  } else if (isNoDetect) {
    // Object out of range — clear the window
    graceActive  = false;
```

```
      graceStartMs = 0;
    } else {
      // Invalid reading (timeout or out-of-sensor-range) — treat as
      // no detection: reset the grace window so a spurious gap
      // does not allow accumulation to continue silently.
      graceActive  = false;
      graceStartMs = 0;
    }
}

// — setup() ————————————————————————————————————
// One-time initialisation called by the Arduino runtime before
// loop() begins.
//
// Sequence:
//  1. Open the Serial port at 9600 baud for diagnostic output.
//  2. Configure all peripheral pin modes.
//  3. Initialise the ST7735S TFT display via hardware SPI.
//  4. Check the TFT sentinel (width == 0 means init failed).
//  5. Disable text wrap so long strings do not refllow.
//  6. Enter the initial DISARMED state.
//
// Requirements: 1.1, 8.1, 8.2, 8.5, 10.1, 11.1–11.6
void setup() {
  // — 1. Serial port ———————————————————————————
  Serial.begin(9600);

  // — 2. Peripheral pin modes ——————————————————————
```

```
  pinMode(PIN_TRIG,   OUTPUT);
  pinMode(PIN_ECHO,   INPUT);
  pinMode(PIN_LED_R,  OUTPUT);
  pinMode(PIN_LED_G,  OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);

  // — 3. TFT initialisation ————————————————
  tft.initR(INITR_BLACKTAB);

  // — 4. TFT init sentinel check ———————————————
  // Adafruit_ST7735::width() returns 0 when initR() did not
  // set up the display geometry, indicating a hardware fault.
  // The system continues with LED / buzzer feedback only.
  if (tft.width() == 0) {
    Serial.println("TFT INIT FAILED");
    // continue — LEDs and buzzer still work
  }

  // — 5. Disable text wrap ————————————————————
  tft.setTextWrap(false);

  // — 6. Enter initial state ——————————————————
  enterState(DISARMED);
}

// — loop() ———————————————————————————————————
// Main execution loop — runs continuously after setup().
```

```
// Acts as a pure dispatcher: calls each subsystem handler
// in order and returns immediately.  All timing, state
// transitions, and output changes are handled inside the
// called functions.
//
// Call order:
//   1. checkButton()       — debounced button edge detection
//   2. updateSensor()      — HC-SR04 poll (ARMED state only)
//   3. updateStateMachine()  — grace period, alarm patterns,
//                              auto-silence
//
// Requirements: 12.5
void loop() {
  checkButton();
  updateSensor();
  updateStateMachine();
}
```